Chapter 5
Apache Portable Runtime (apr)

Abstract In this chapter we discuss the Apache Portable Runtime (APR) application development framework library API. In particular we discuss the APR memory pool, process, thread, and thread pool. We present example which use these functions in a real-life setting. APR file information functions are used with memory mapped IO. APR hash tables are used to develop a word frequency counting applications. We use the Memcache library with APR, and also present an example which uses APR shared memory.

Contents

5.1 APR Memory Pool **114**
5.2 APR Processes **114**
5.3 APR Threads **116**
5.4 APR Thread Pool **117**
5.5 File information, IO, and Memory mapped files **119**
5.6 Hash tables **120**
5.7 Using Memcache with APR **122**
5.8 Shared memory with APR **124**
5.9 Conclusion **126**

The Apache Portable Runtime was written to support the portability of the Apache HTTP web server, and as such provides an operating system abstraction. The Subversion (SVN) version control system also uses APR. Another application development framework is Nokia/Trolltech Qt Framework (see Section 19.1.3 for more details). APR can be used to provide the application with basic functionalities in a consistent API on multiple platforms. The main functionalities of APR are shown below:

1. Memory pool,
2. Atomic operations: interface to *atomics* (see Section 3.1.7.1),
3. Dynamic Object handling,
4. Environment functions,
5. Signal handling,
6. File information and IO,

S. Koranne, *Handbook of Open Source Tools*,
DOI 10.1007/978-1-4419-7719-9_5, © Springer Science+Business Media, LLC 2011

7. Hash tables,
8. Memory map and allocation, shared memory, network library,
9. Thread and process library,
10. Mutex and condition variables, read/writer locks.

These features are described in detail with the help of examples in this chapter.

5.1 APR Memory Pool

The distinct advantages of using a memory pool over standard `malloc`, **`new`** are the following:

1. Efficiency: a custom memory pool can be more efficient than standard library implementation,
2. Resource tracking: a memory pool can be used as a dynamic heap where the life time of memory is tracked. Once the pool is destroyed all the memory is automatically reclaimed,
3. Constructor/Destructor: APR memory pools provide raw memory bits, so cost of constructor is not added (caveat, neither is its convenience of initializing complex data structures).

The key functions for APR memory pools are:

```
apr_pool_create( apr_pool_t **pool, apr_pool_t *parent);
apr_allocator_t * apr_pool_allocator_get( apr_pool_t*);
apr_allocator_max_free_set( allocator, 64 );
void* apr_palloc( apr_pool_t *pool, apr_size_t size);
apr_pool_clear( apr_pool_t *p);
apr_pool_destroy( apr_pool_t *p );
```

Using a parent pool we can build *hierarchy* of pools, such that when the parent pool is destroyed all child pools are destroyed as well. It is also possible to register a *callback* function to be called when a pool is destroyed. Examples of using APR memory pools are presented in the next section.

5.2 APR Processes

Consider the program listing as shown in Listing 5.1.

```
// \file calendar_proc.pp
// \author Sandeep Koranne (C) 2010
// \description Example of APR process type
#include <iostream>
#include <stdlib.h>
#include <assert.h>
#include <apr_general.h>
#include <apr_thread_proc.h>

static const char *PROGRAM_ARG[ 16 ];
```

```
int main( int argc, char *argv[] ) {
  apr_status_t retval;
  apr_pool_t   *pool;
  apr_initialize();
  apr_pool_create( &pool, NULL );
  apr_procattr_t *attribute;
  /* first create the attribute */
  retval = apr_procattr_create( &attribute, pool );
  retval = apr_procattr_io_set( attribute, APR_NO_PIPE,
        APR_FULL_BLOCK, APR_NO_PIPE );
  //retval = apr_procattr_cmdtype_set( attribute, APR_PROGRAM_ENV );
  retval = apr_procattr_cmdtype_set( attribute, APR_PROGRAM_PATH );
  apr_proc_t calendar_process;
  PROGRAM_ARG[0] = "cal";
  PROGRAM_ARG[1] = "1";
  PROGRAM_ARG[2] = "2010";
  PROGRAM_ARG[3] = NULL;
  retval = apr_proc_create( &calendar_process, PROGRAM_ARG[0],
          PROGRAM_ARG, NULL, attribute, pool );
  if( retval != APR_SUCCESS ) exit( 1 ); //
  while( true ) { // read data from child process
    char buf[ 1024];
    retval = apr_file_gets( buf, sizeof( buf ), calendar_process.out );
    if( APR_STATUS_IS_EOF( retval ) ) break;
    std::cout << buf;
  }
  apr_file_close( calendar_process.out );
  int status;
  apr_exit_why_e why;
  retval = apr_proc_wait( &calendar_process, &status, &
        why, APR_WAIT );
  if( APR_STATUS_IS_CHILD_DONE( retval ) ) {
    std::cout << "WHY = ";
    switch( why ) {
    case APR_PROC_EXIT:
      { std::cout << "APR_PROC_EXIT"; break; }
    case APR_PROC_SIGNAL:
      { std::cout << "APR_PROC_SIGNAL"; break; }
    case APR_PROC_SIGNAL_CORE:
      { std::cout << "APR_PROC_SIGNAL_CORE"; break; }
    }
    std::cout << " Status = " << status;
  } else {
    std::cout << "still processing...";
  }
  std::cout << std::endl;
  apr_terminate();
  return (0);
}
```

Listing 5.1 APR process running 'cal' program

In the above listing we see the use of several APR functions and data structures. Before using APR functionality, APR has to be initialized using: `apr_initialize()`. APR has a memory pool functionality which is used by most of APR's functions, as well as application programs. We create a memory pool using `apr_pool_create` function. The next several lines are used to create process attributes which control the process's behavior, detach state, and IO. In this case we want to read from the child process output, thus we use `APR_FULL_BLOCK` for its output.

We *wait* for the process to terminate (otherwise we would create a *zombie* process), and print its exit code and reason for exiting.

5.3 APR Threads

In addition to supporting process based multi-tasking, APR also has support for multi-threading as shown in Listing 5.2. In this program we create a thread which prints a message. The thread creation function in APR is shown below:

```
/**
 * Create a new thread of execution
 * @param new_thread The newly created thread handle.
 * @param attr The threadattr to use to determine how to create the thread
 * @param func The function to start the new thread in
 * @param data Any data to be passed to the starting function
 * @param cont The pool to use
 */
apr_thread_create(apr_thread_t **new_thread,
                  apr_threadattr_t *attr,
                  apr_thread_start_t func,
                  void *data, apr_pool_t *cont);
```

```
// \file tapr.cpp
// \author Sandeep Koranne, (C) 2010
// APR test and thread support

#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <apr_general.h>
#include <apr_thread_proc.h>

#define NUM_THREADS 4

static void* APR_THREAD_FUNC comp( apr_thread_t *T, void* arg);

int main( int argc, char *argv [] ) {
  apr_status_t retval;
  apr_pool_t   *pool;
  apr_thread_t *THREADS[ NUM_THREADS ];
  apr_threadattr_t *thread_attribute;
  int i,j;

  apr_initialize();
  apr_pool_create( &pool, NULL );
  apr_threadattr_create( &thread_attribute, pool );
  for(i=0; i < NUM_THREADS; ++i ) {
    retval = apr_thread_create( &THREADS[i], thread_attribute,
        comp, NULL, pool );
  }
  for(i=0; i < NUM_THREADS; ++i ) {
    retval = apr_thread_join( &retval, THREADS[i] );
  }
  apr_terminate();
  return 0;
}

static void* APR_THREAD_FUNC comp( apr_thread_t *T, void* arg ) {
  printf("\n Inside apr_thread function: comp");
  apr_thread_exit( T, APR_SUCCESS );
  return NULL;
}
```

Listing 5.2 APR thread example

5.4 APR Thread Pool

Scheduling jobs such that CPUs are efficiently utilized is a non-trivial problem when the problem is not uniform, and has data dependent runtime. In such situation it is often easier to design a pool of threads which can be dynamically assigned to perform computation. Towards this end, APR has a *thread pool* facility. An example of a thread pool to perform compute intensive calculation is shown in Listing 5.3.

```
/* \file thpool.cpp
   \author Sandeep Koranne, (C) 2010
   Fibonacci = (0,1,1,2,3,5,8,13,21,34,55,89)
   Sum = prefix-sum of F
   (0,1,2,4,4,12,20,
*/
#include <stdio.h>
#include <pthread.h>
#include <string.h>
#include <stdlib.h>
#include <apr_general.h>
#include <apu.h>
#include <apr_reslist.h>
#include <apr_thread_pool.h>
#include <cassert>
#include <iostream>

struct Data {
  int s0, s1, N;
  unsigned int sum;
};

unsigned int CountCollatzReturn( unsigned int N ) {
  unsigned int count = 0;
  while( 1 ) {
    count++;
    if( N <= 1 ) return count;
    if( N % 2 ) N = 3*N+1;
    else N = N/2;
  }
  return count;
}

#if 0
/**
   \function SumFibonacci
   @param s0, s1 contain initial fibonacci numbers
   @param N represents how many numbers to sum
   @return sum of N fibo numbers from (s0,s1)
   \description Algorithm is non-recursive
*/
unsigned int SumFibonacci( int s0, int s1, int N ) {
  int i,j;
  unsigned int next=s0+s1;
  unsigned int sum=0;
  for(i=0; i < N-2; ++i ) {
    next = s0 + s1;
    s0 = s1;
    s1 = next;
    sum += next;
  }
  return sum ;
}
#else
unsigned int SumFibonacci( int s0, int s1, int N ) {
  std::cout << "Running function " << __FUNCTION__ << std::endl;
```

```
  int i,j;
  unsigned int next=s0+s1;
  unsigned int sum=0;
  for(j=0; j < 100000000; ++j ) {
    sum = 0;
    for(i=0; i < N-2; ++i ) {
      next = s0 + s1;
      s0 = s1;
      s1 = next;
      sum += next;
    }
  }
  return sum ;
}
#endif

void* APR_THREAD_FUNC FUNCTION( apr_thread_t *thd, void* arg ) {
  struct Data* data = (struct Data*) arg;
  data->sum = SumFibonacci(data->s0, data->s1, data->N );
  data->sum = CountCollatzReturn( data->sum );
  return APR_SUCCESS;
}

//#define SERIAL_VERSION
#ifdef SERIAL_VERSION
int main( int argc, char * argv[] ) {
  apr_status_t retval;
  apr_pool_t *pool;

  const int LOOP_COUNT  = 8;
  const int Z = 7;
  int s0=0, s1=1;
  int i,j,rc=0;
  unsigned int sum = 0;
  struct Data data[LOOP_COUNT];
  for( i=0; i < LOOP_COUNT; ++i ) {
    data[i].s0 = i; data[i].s1 = i+1; data[i].N = (i*Z);
  }

  apr_initialize();
  apr_pool_create( &pool, NULL );
  for(j=0; j < LOOP_COUNT; ++j ) {
    FUNCTION( NULL, (void*) &data[j] );
  }
  // all work has been done
  for( i=0; i < LOOP_COUNT; ++i ) {
    printf("\n sum (%d,%d,%d) = %d", i,(i+1),(i*Z), data[i].sum );
  }
  printf("\n");
  return ( EXIT_SUCCESS );
}
#endif

#define THREADED_VERSION
#ifdef THREADED_VERSION
int main( int argc, char * argv[] ) {
  apr_status_t retval;
  apr_pool_t *pool;

  const int LOOP_COUNT  = 8;
  const int Z = 7;
  int s0=0, s1=1;
  int i,j,rc=0;
  unsigned int sum = 0;
  struct Data data[LOOP_COUNT];
  for( i=0; i < LOOP_COUNT; ++i ) {
    data[i].s0 = i; data[i].s1 = i+1; data[i].N = (i*Z);
```

```
  }

  apr_thread_pool_t *thrp;
  apr_initialize();
  apr_pool_create( &pool, NULL );
  retval = apr_thread_pool_create( &thrp, 2, 4, pool );
  if( retval != APR_SUCCESS ) assert( 0 && "apr_thread_pool_create");


  for( j=0; j < LOOP_COUNT; ++j ) {
    retval = apr_thread_pool_push( thrp, FUNCTION, &data[j], 0, NULL );
    if( retval != APR_SUCCESS )
      assert( 0 && "apr_thread_pool_push");
  }

  apr_size_t scount = apr_thread_pool_scheduled_tasks_count( thrp );
  std::cout << std::endl << scount
            << " tasks have been scheduled..." << std::endl;
  scount = apr_thread_pool_tasks_count( thrp );
  std::cout << std::endl << scount
            << " tasks are waiting..." << std::endl;
  scount = apr_thread_pool_tasks_run_count( thrp );
  std::cout << std::endl << scount
            << " tasks have completed..." << std::endl;

  while( scount != LOOP_COUNT ) {
    scount = apr_thread_pool_tasks_run_count( thrp );
    if( scount == LOOP_COUNT ) break;
    std::cout << std::endl << scount
              << " tasks have completed..." << std::endl;
    sleep(5);
  }
  // all work has been done
  retval = apr_thread_pool_destroy( thrp );
  for( i=0; i < LOOP_COUNT; ++i ) {
    printf("\n sum (%d,%d,%d) = %d", i,(i+1),(i*Z), data[i].sum );
  }
  printf("\n");
  return ( EXIT_SUCCESS );
}
#endif
```

Listing 5.3 APR thread pool example

5.5 File information, IO, and Memory mapped files

APR supports an abstract file type, `apr_file_t` as well as a *file info* type, `apr_finfo_t`. To open a file for reading we can use the following APR function:

```
  apr_status_t retval;
  apr_pool_t *pool;
  apr_pool_create( &pool, NULL );
  apr_finfo_t file_info;
  apr_file_t  *fp;
  retval = apr_file_open( &fp, fileName,
        APR_READ|APR_BINARY,
        APR_OS_DEFAULT, pool );
  retval = apr_file_info_get( &file_info, APR_FINFO_SIZE, fp );
```

Moreover, memory mapped files can be created using the `apr_mmap_create` function, e.g.:

```
retval = apr_mmap_create( &mmap, fp, 0, file_info.size,
        APR_MMAP_READ, pool );
// .. use the mmap->mm data as const char *
apr_mmap_delete( mmap ); // unmap the file
```

5.6 Hash tables

Hash tables in APR are implemented using APR *pools*. The following functions are useful when developing applications using APR hash tables:

```
apr_hash_t* apr_hash_make( apr_pool_t* );
void* apr_hash_get( apr_hash_t*, void* key, apr_ssize_t keylen, void* val);
```

A simple word frequency counting application using APR hash tables is presented in Listing 5.4.

```
// \file apr_hash_example.cpp
// \author Sandeep Koranne (C) 2010
// \description Use of hash table in APR
#include <iostream>           // Program IO
#include <fstream>            // File IO
#include <cstdlib>            // exit
#include <string>             // STL string
#include <sstream>            // string iterator
#include <cassert>            // assertion checking
#include <cstring>            // memory functions
#include <apr_general.h>      // APR basic functions
#include <apr_hash.h>         // APR hash tables

static int PrintHashFunction( void* dummy, const void* key,
                              apr_ssize_t klen, const void* val ) {
  std::cout << (const char*)key << "\t" << (int)val << "\n";
}

static void CountWordFrequency( const char* filename ) {
  std::ifstream ifs( filename );
  if( !ifs ) {
    std::cerr << "Unable to open file: " << filename;
    exit(1);
  }
  apr_status_t retval;
  apr_pool_t *pool;
  apr_pool_create( &pool, NULL );
  apr_hash_t *word_hash = apr_hash_make( pool );
  assert( word_hash && "Word hash not created." );
  char line[1024];
  while( ifs ) {
    ifs.getline( line, 1024 );
    std::istringstream sstr( line );
    while( sstr ) {
      std::string word; sstr >> word;
      if( word == "" ) break;
      std::cout << "Processing " << word << " " << word.size() << "\n";
      char *next_word = new char[ word.size()+1];
      strcpy( next_word, word.c_str() );
      void* val = apr_hash_get( word_hash, (void*) next_word, word.size() );
```

```
      if( val == NULL ) { // first time
        apr_hash_set( word_hash, (void*) next_word, word.size(),
          (const void*)(1) );
      } else {
        int count = int(val) + 1;
        apr_hash_set( word_hash, (void*) next_word, word.size(),
          (const void*)(count) );
      }
    }
  }
  // now the hash table has been constructed.
  std::cout << "Hash has " << apr_hash_count( word_hash ) << " elements.\n";
  {
    apr_hash_index_t *hi;
    for (hi = apr_hash_first(pool, word_hash); hi; hi = apr_hash_next(hi)) {
      void *val;
      const void *key_word;
      apr_ssize_t key_len;
      apr_hash_this(hi, &key_word, &key_len, &val);
      std::cout << (const char*)(key_word) << "\t" << (int)val << "\n";
    }
  }
  //(void)apr_hash_do( PrintHashFunction, NULL, word_hash );
  apr_pool_destroy( pool );
}

int main( int argc, char *argv [] ) {
  if( argc != 2 ) {
    std::cerr << "Usage: ./apr_hash_example <file>..\n";
    exit(1);
  }
  apr_initialize();
  CountWordFrequency( argv[1] );
  std::cout << std::endl;
  apr_terminate();
  return (0);
}
```

Listing 5.4 Example using APR hash tables for word frequency counting

In Listing 5.4 it should be noted that the key to the hash table is the address of an allocated memory containing the word. Although, the hash function correctly calculates the hash for different words, reusing the same address as the key for different words will not work, as the hash table does not store keys inside the hash table (it only stores the value for that key, indexed by the computed hash function). Compiling the program shown in Listing 5.4 and running it on a small data set gives us:

```
./apr_hash_example small_words.txt
Processing jill 4
Processing the 3
Processing jack 4
Processing jack 4
Processing jill 4
Hash has 3 elements.
the 1
jack 2
jill 2
```

The program (in Listing 5.4) also shows the two hash table iteration mechanisms available in APR; (i) the use of the hash index type and (ii) using the callback function.

5.7 Using Memcache with APR

As we will discuss in Section 9.6, *memcached* is a high-performance distributed memory object caching system. It is designed for reducing the database load in web applications and speed up dynamic web content generation. It comprises of a server component which manages the lifetime of the cached objects which can be accessed on the network. Using APR, we can integrate Memcache functionality in an application. Ofcourse, a valid running `memcached` process should be executing on the server with the port accessible to the client program. An example of using APR with Memcache is shown in Listing 5.5.

```
// \file aprcache.cpp
// \author Sandeep Koranne (C) 2010
// \description Use of memcache server with APR
#include <iostream>           // Program IO
#include <cassert>            // assertion checking
#include <cstdlib>            // exit
#include <fstream>            // C++ STL for file IO
#include <apr_general.h>      // APR functions
#include <apr_memcache.h>     // memcache client in APR
#if 0
echo "stats settings" | nc localhost 11211
#endif
int main( int argc, char *argv []) {
  apr_initialize();
  apr_status_t retval;
  apr_pool_t *pool;
  apr_pool_create( &pool, NULL );
  // create a server object
  apr_memcache_server_t *myserver;
  retval = apr_memcache_server_create( pool, "localhost", 11211,
                                       1, 10, 10, 1000, &myserver );
  if( retval != 0 )
    { std::cerr << "Unable to create server object..\n"; exit(1); }

  apr_memcache_t *mycache;
  retval = apr_memcache_create( pool, 2, 0x0, &mycache );
  if( retval != 0 ) exit(1);

  retval = apr_memcache_add_server( mycache, myserver );
  if( retval != 0 )
    { std::cerr << "Unable to add server object..\n"; exit(1); }

  retval = apr_memcache_enable_server( mycache, myserver );
  if( retval != 0 )
    { std::cerr << "Unable to enable server object..\n"; exit(1); }

  #if 0
  retval = apr_memcache_set( mycache, "ABCDE", "KEY", 3, 100, 0x0 );
  if( retval != 0 )
    { std::cerr << "Unable to store data on server object..\n"; exit(1); }
  #endif
  // now check to see if you can retrieve this information
  char *baton = NULL;
```

```
  apr_size_t baton_len = 0;
  retval = apr_memcache_getp( mycache, pool, "ABCDE",
             &baton, &baton_len, 0x0 );
  // An application which uses memcache must be prepared
  // to assume data is not present on the server.
  if( ( retval != 0 ) || ( baton == NULL ) || ( baton_len == 0 ) ) {
    std::cout << "Data for key ABCDE does not exist in server.\n";
  } else {
    std::cout << "Memcache server returned : " << baton << std::endl;
  }

  apr_memcache_server_t *server =
    apr_memcache_find_server( mycache, "localhost", 11211 );
  if( server ) {
    std::cout << "Found memcached server running...\n";
  } else {
    std::cout << "Unable to find memcached server running...\n";
    exit(1);
  }

  // get server statistics
  apr_memcache_stats_t *stats;
  retval = apr_memcache_stats( myserver, pool, &stats );
  if( retval != 0 ) {
      std::cerr << "Unable to collecte memcache server stats..\n";
      exit( 1 );
    }
  std::cout << "\nMemcache server stats...." << std::endl
            << "\nVersion information      : " << stats->version
            << "\nPID                      : " << stats->pid
            << "\nUptime                   : " << stats->uptime
            << "\nCurrent items stored     : " << stats->curr_items
            << "\nBytes used               : " << stats->bytes
            << "\nCurrent connections      : " << stats->curr_connections
            << "\nBytes written            : " << stats->bytes_written;

  std::cout << std::endl;
  apr_terminate();
  return (0);
}
```

Listing 5.5 Using memcache with APR

Compiling and running this program gives us:

```
./aprcache
Memcache server returned : KEY
Found memcached server running...
Memcache server stats....
Version information      : 1.4.5
PID                      : 30661
Uptime                   : 1920
Current items stored     : 1
Bytes used               : 57
Current connections      : 10
Bytes written            : 9251

$ ./aprcache
Data for key ABCDE does not exist in server.
Found memcached server running...
Memcache server stats....
Version information      : 1.4.5
```

```
PID                       : 30661
Uptime                    : 1991
Current items stored      : 0
Bytes used                : 0
Current connections       : 10
Bytes written             : 10050
```

It can be seen on line 38 of Listing 5.5 that we install the data on the server with a lifetime of 100 seconds. This means, that after 100 seconds, the data is ejected from the cache. By compiling the program with and without the data insertion we can see the effect of cache ejection as shown in the above transcript.

5.8 Shared memory with APR

As we have seen above APR has functions for invoking sub-processes and multi-processing. Inter-process communication involving large shared data segments is often accomplished using *shared memory*. Shared memory segments are memory locations which can be accessed by multiple processes. All the processes must agree (or be informed of) the *segment name* through which the shared memory will be access. As with any shared resource, critical sections need to be protected against inadvertent use by another process. The APR functions `apr_shm_create` creates a shared segment of the given name and size. Function, `apr_shm_destroy` destroys the shared segment. During the time the shared segment is active, its memory address can be calculated using the `apr_shm_baseaddr_get` function which returns a handle to the underlying memory. An example of creating a shared memory segment is shown in Listing 5.6.

```
// \file aprshmA.cpp
// \author Sandeep Koranne, (C) 2010
// \description Example of shared memory using APR
#include <iostream>           // for program IO
#include <cstdlib>            // for exit
#include <apr_general.h>      // APR functions
#include <apr_shm.h>          // APR shared memory

static const char SHM_FILE_NAME[] = "SHM1234MHS"; // should be common

int main( int argc, char *argv [] ) {
  apr_initialize();
  apr_status_t retval;
  apr_pool_t *pool;
  apr_pool_create( &pool, NULL );
  apr_shm_t *shm_segment;
  retval = apr_shm_create( &shm_segment, 1024, SHM_FILE_NAME, pool );
  if( retval != 0 ) {
    std::cerr << "Unable to create shared memory segment..\n";
    exit( 1 );
  }
  int *data = (int*) apr_shm_baseaddr_get( shm_segment );
  for( int i=0; i < 10; ++i ) data[i] = i;

  int x;
  std::cout << "Enter a number when done with programB:";
  std::cin >> x;
```

```
  std::cout << std::endl;

  apr_shm_destroy( shm_segment );
  apr_pool_destroy( pool );
  apr_terminate();
  return (0);
}
```

Listing 5.6 Example of using shared memory with APR

The other process requests access to the shared segment using the `apr_shm_attach` function using the same filename as the process which created the shared segment. Once the segment has been attached the baseaddress and segment size can be inspected. An example of creating a shared memory segment is shown in Listing 5.7.

```
// \file aprshmA.cpp
// \author Sandeep Koranne, (C) 2010
// \description Example of shared memory using APR
#include <iostream>           // for program IO
#include <cstdlib>            // for exit
#include <apr_general.h>      // APR functions
#include <apr_shm.h>          // APR shared memory

static const char SHM_FILE_NAME[] = "SHM1234MHS"; // should be common

int main( int argc, char *argv [] ) {
  apr_initialize();
  apr_status_t retval;
  apr_pool_t *pool;
  apr_pool_create( &pool, NULL );
  apr_shm_t *shm_segment;
  retval = apr_shm_attach( &shm_segment, SHM_FILE_NAME, pool );
  if( retval != 0 ) {
    std::cerr << "Unable to attach shared memory segment..\n";
    exit( 1 );
  }
  apr_size_t data_size = apr_shm_size_get( shm_segment );
  std::cout << "size of shared memory segment = " << data_size << "\n";
  int *data = (int*) apr_shm_baseaddr_get( shm_segment );
  for( int i=0; i < 10; ++i ) {
    if( data[i] != i ) {
      std::cerr << "Data corrupted in shared segment..\n";
      exit(1);
    }
  }
  std::cout << std::endl << "Data in shared segment is valid.\n";
  apr_shm_detach( shm_segment );
  apr_pool_destroy( pool );
  apr_terminate();
  return (0);
}
```

Listing 5.7 Using shared memory segment with APR

The Qt library also has shared memory segment functionality available as part of its application programming API, see Section 19.1.4.1 for more details. For POSIX shared memory functionality, run `man shm_overview` for an overview of the shared memory functions in POSIX.

5.9 Conclusion

Apache Portable Runtime (APR) is indeed a versatile application development library. It not only provides an operating system independent set of routines with predetermined semantics on a wide variety of machine, OS combinations, but also provides useful application development framework utilities such as hash tables, memory pools, multi-threading and multi-process supports, memory mapping, and network access.